A3: Style Transfer

Andrew Vasquez (avasquez56@csuchico.edu)
CSCI 611 - Applied Machine Learning Summer 2025
Bo Shen
2025-Jul-06

Purpose

The purpose of this assignment is to familiarize ourselves with an implementation of the Visual Geometry Group (VGG) models, specifically VGG19 based on the work defined in Gatys 'Image Style Transfer Using Convolutional Neural Networks' research work in 2016. Tooling used for exercise included Jypyter Labs/Notebook with pre-trained VGG models defined in the Python TourchVision libraries.

The oil panting in canvas work titled *'Composition II in Red, Blue, and_Yellow'* by Piet Mondriaan (1930) is used as the Style image. The renaissance painting *'Lamentation of Christ'* by Andrea Mantegna (1475) is used as the content image. These images were chosen based to demonstrate the effect of abstractionism on a detailed, though proportionally incorrect, painting.

To minimize computation latency, both content and style images were down-sampled (to ~400x~400 pixels) prior to IPNYB-usage. This pre-down-sampling did not provide a measurable decrease in latencies, as the load_image() function effectively duplicated this effort at run-time.

Layer definitions are based on layer cross-sections of the generic VGG model, specifically:

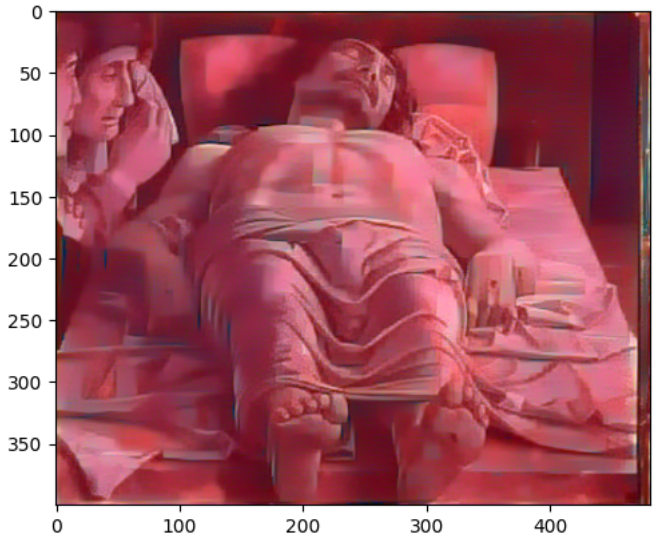
```
## VGG19 layer names from print(vgg) [above]
```

```
layers = {'0': 'conv1_1',
    '5': 'conv2_1',
    '10': 'conv3_1',
    '19': 'conv4_1',
    '21': 'conv4_2',  ## content layer
    '28': 'conv5_1'}
```

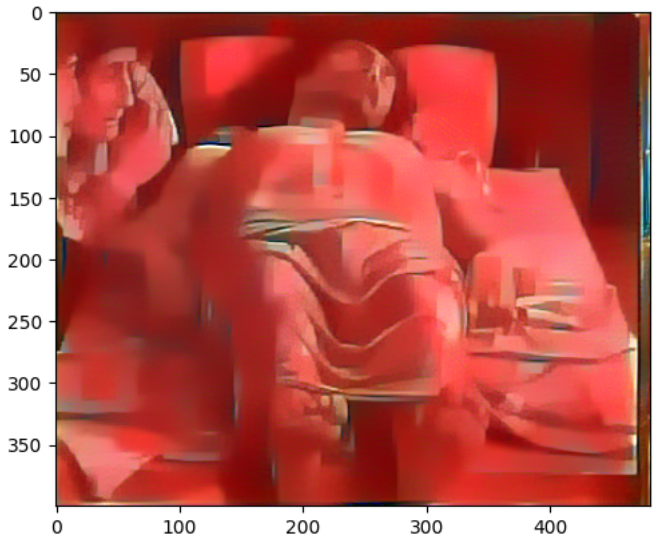
Computations for the Gram matrix follow the pre-scribed calculations -- including dimension flattening and reshaping against the flattened transpose.

Retaining the 2000 step sampling (five total snapshots) resulted in computational times of more than ~XX minutes, leading to these stylized, proportional results:

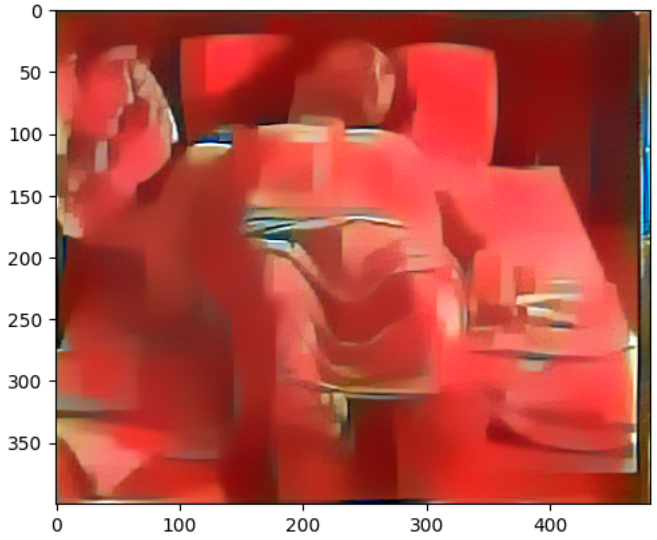
```
Total loss:  162370384.0
```



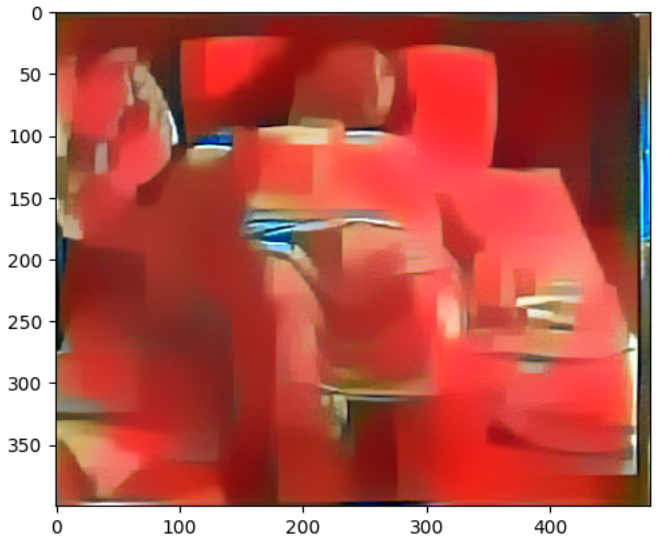

```
Total loss:  42980620.0
```



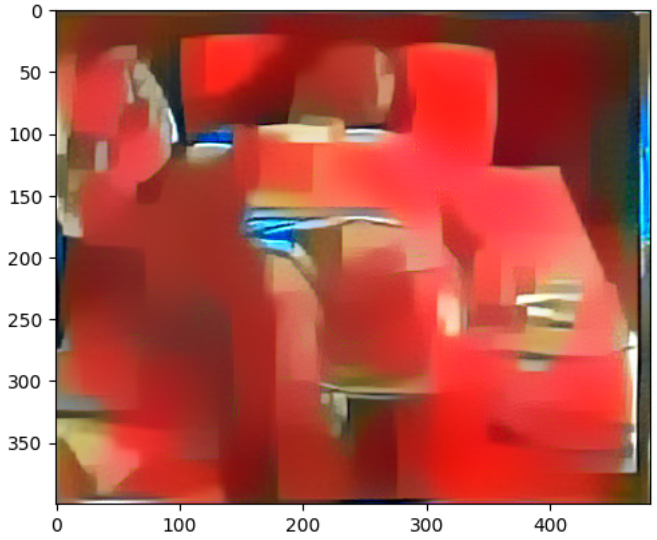

```
Total loss:  30242846.0
```




```
Total loss:  22293148.0
```




```
Total loss:  16202852.0
```

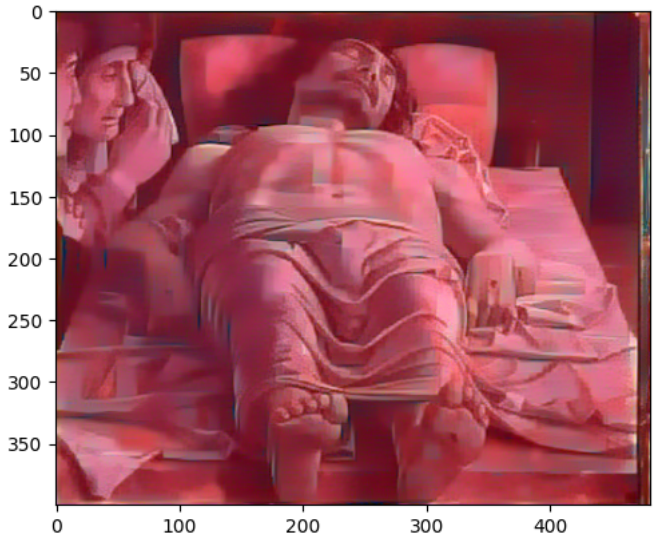


Hyperparameter adjustment, seen below, include reducing steps sampling to 800 (reducing runtimes to ~8 minutes with two snapshots), with content image weight variations of .5 and 1, and style image weight variations of 1e6 and 1e2:

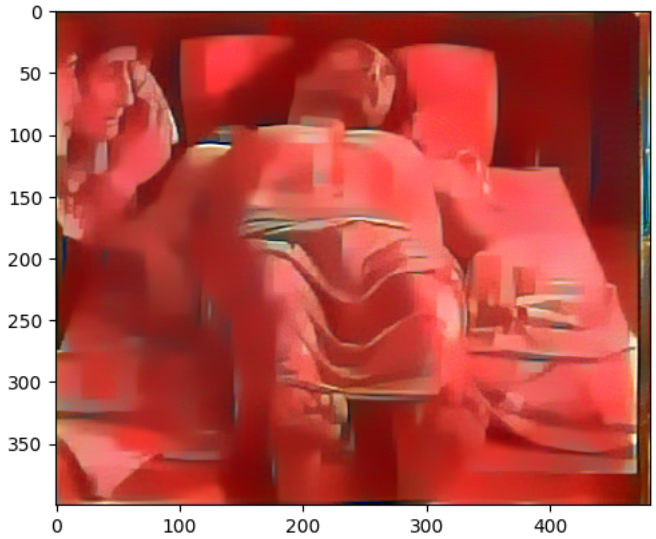
content_weight = 1 # alpha

style_weight = 1e6 # beta

```
Total loss:  162370384.0
```



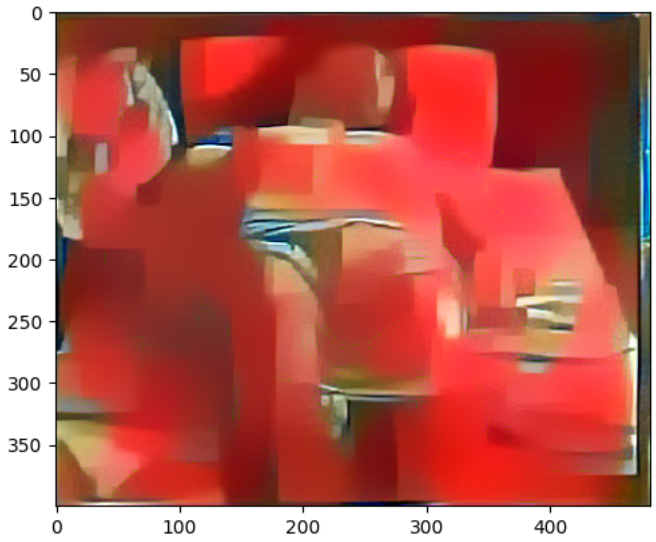

```
Total loss:  42980620.0
```



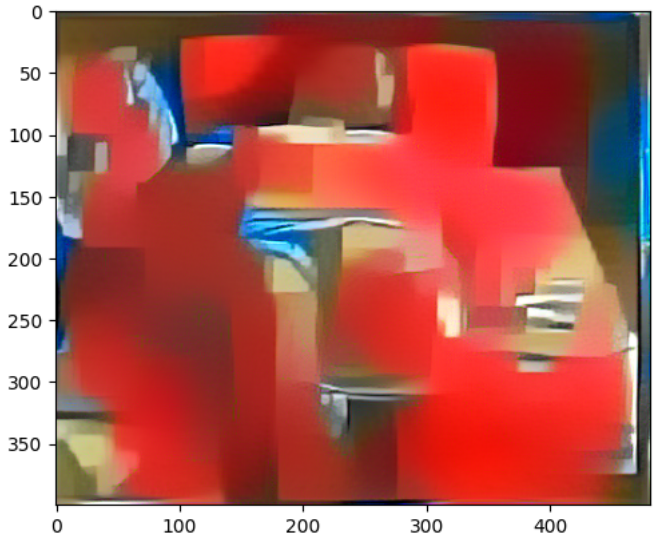
content_weight = 1 # alpha

style_weight = 1e2 # beta

```
Total loss:  2212.270263671875
```



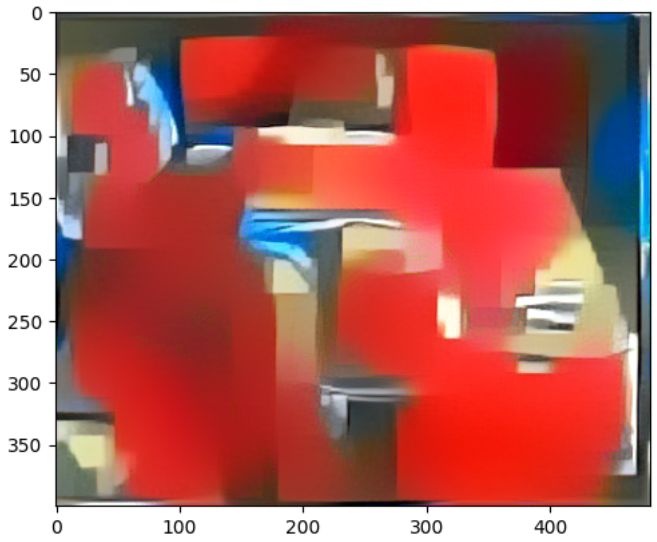
Total loss: 1141.564697265625



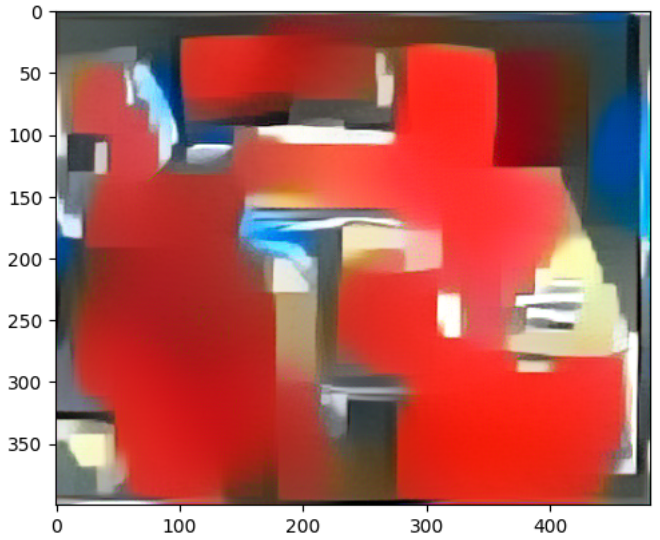
content_weight = .5 # alpha

style_weight = 1e6 # beta

Total loss: 7521933.0



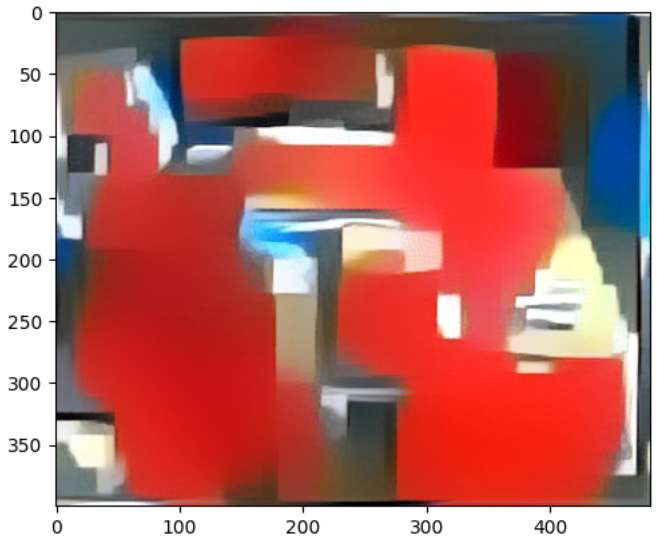
Total loss: 5430076.5



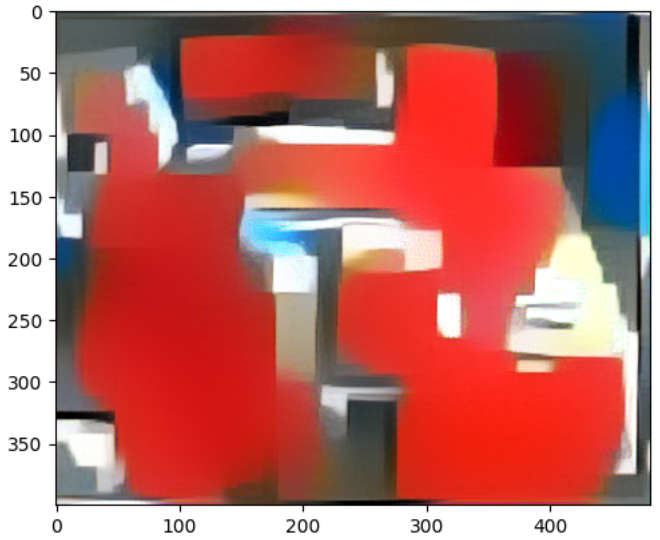
content_weight = .5 # alpha

style_weight = 1e2 # beta

Total loss: 435.5289001464844




```
Total loss:  343.5267028808594
```



Summary

As one can expect modification to these 'Alpha' / 'Beta' adjustments demonstrate how one can sample VGG convolution at distinct times. Computational times can be significantly reduced by execution on GPU-capable systems. Though I imagine a future assignment will be tasked to utilize 'Depthwise Separable Convolution', to reduce computation overhead within layers.